

Tutorium 1

Endrekursion vs. Lineare Rekursion

Grundlagen der Praktischen Informatik

15. Juni 2026

? Typische Frage

Bewerten Sie, ob die folgende Aussage korrekt ist und begründen Sie Ihre Antwort:

„Jede endrekursive Funktion ist auch linear rekursiv umsetzbar.“

❓ Typische Frage

Bewerten Sie, ob die folgende Aussage korrekt ist und begründen Sie Ihre Antwort:

„Jede endrekursive Funktion ist auch linear rekursiv umsetzbar.“

✓ **Kurzantwort:** Ja!

- ▶ Jede endrekursive Funktion lässt sich in eine (äquivalente) linear rekursive Funktion umschreiben.

Endrekursion → Lineare Rekursion

Endrekursion: Der rekursive Aufruf ist der letzte Ausdruck. Kein weiterer Work nach der Rückkehr!

Endrekursiv (mit Akkumulator)

```
1 func fak(n, acc):  
2     if n == 0:  
3         return acc  
4     return fak(n-1, n *  
        acc)
```

Aufruf: fak(3, 1) → fak(2, 3) → ...
→ 6

Linear Rekursiv

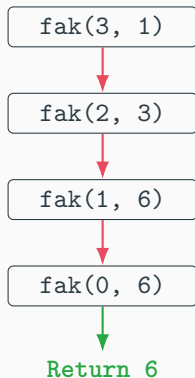
```
1 func fak(n):  
2     if n == 0:  
3         return 1  
4     return n * fak(n-1)
```

Aufruf: 3 * fak(2) → 3 * 2 * fak(1)
→ ... → 6

Visueller Vergleich: Call Stack

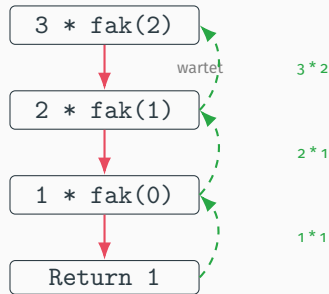
Endrekursiv (Tail Call)

Der Zustand wird im `acc` weitergegeben.
Kein wachsender Stack!



Linear Rekursiv

Jeder Aufruf wartet auf das Ergebnis. Der Stack wächst!



Lineare Rekursion → Endrekursion

Die Umwandlung von linear zu endrekursiv ist **nicht immer trivial**. Es hängt davon ab, ob das Ergebnis schrittweise als Akkumulator aufgebaut werden kann.

Beispiel: $f(x)$

```
1 func f(x):  
2     if x == 0:  
3         return 1  
4     return 2 * f(x-1) + 1
```

Problem: Das Ergebnis ist auf jeder Ebene eine lineare Kombination. Eine direkte Endrekursion ohne Hilfsmittel (geschlossene Form oder iterative Akkumulatoren) ist nicht offensichtlich.

Lösung über die geschlossene Form

Oft hilft eine Umformung (geschlossene Form). Für unser Beispiel gilt:

$$f(x) = 2^{x+1} - 1$$

Iterative/Endrekursive Umsetzung

```
1 func f_tail(x, acc):  
2     if x == 0:  
3         return acc  
4     return f_tail(x-1, 2*acc + 1)
```

Aufruf für $x = 3$, $acc = 1$:

$f_tail(3, 1) \rightarrow f_tail(2, 3) \rightarrow f_tail(1, 7) \rightarrow f_tail(0, 15) \rightarrow 15$

☰ Wichtige Erkenntnisse

- ▶ **Endrekursion** → **Linear**: Immer möglich! (Akkumulator entfernen, Rechnung beim Zurückkehren durchführen).
- ▶ **Linear** → **Endrekursion**: Nicht immer trivial. Manche linearen Rekursionen benötigen zusätzliche Akkumulatoren oder eine geschlossene Form.
- ▶ **Prinzip**: Endrekursion trägt das Zwischenergebnis in Parametern; lineare Rekursion baut das Ergebnis beim Zurückkehren auf.

Live Demo



ex00.hs

Wir programmieren den **Hello World** und **ggT** (Größter gemeinsamer Teiler)
zusammen im Code!